FLEX / YACC / BISON

Complete Installation Guide

Linux
Ubuntu / Debian / Fedora / Arch

Windows
WSL2 / Cygwin / MSYS2 / WinFlexBison

Covers installation, verification, and a working Hello World example

1. Introduction

Flex (Fast Lexical Analyzer Generator) and Yacc/Bison (Yet Another Compiler Compiler / GNU Parser Generator) are foundational tools in compiler construction, interpreter development, and domain-specific language (DSL) design. Together they form a classic lexer–parser pipeline.

Tool	Purpose
flex	Generates a C lexer (tokenizer) from regular expression rules
lex	Original AT&T lexer; flex is the GNU-compatible replacement
yacc	Original POSIX parser generator; produces LALR(1) parsers in C
bison	GNU replacement for yacc; adds features like GLR parsing and C++ support

The typical workflow: `.l (flex rules)` → `lex.yy.c` and `.y (bison rules)` → `y.tab.c / y.tab.h`, then both compiled with GCC into an executable.

2. Prerequisites

Before installing Flex and Bison, ensure the following are present on your system:

Requirement	Notes
C Compiler (GCC / Clang / MSVC)	GCC is strongly recommended; included in build-essential on Debian/Ubuntu
make	Used to build generated source files; often pre-installed on Linux
Terminal / Shell	Bash (Linux), PowerShell or CMD (Windows), or WSL2 bash
Administrative privileges	Required for package manager commands (sudo on Linux, Admin on Windows)

3. Linux Installation

3.1 Ubuntu / Debian (apt)

Ubuntu 20.04+, Debian 11+ and their derivatives all use apt as their package manager.

Step 1 — Update package index

```
sudo apt update && sudo apt upgrade -y
```

Step 2 — Install build tools

```
sudo apt install -y build-essential
```

Step 3 — Install flex and bison

```
sudo apt install -y flex bison
```

Step 4 — Verify installation

```
flex --version
bison --version
gcc --version
```

Expected output (versions may vary)
flex 2.6.4 / GNU Bison 3.8.2 / gcc 11.x.x

3.2 Fedora / RHEL / CentOS (dnf / yum)

Step 1 — Update packages

```
sudo dnf update -y
```

Step 2 — Install development tools

```
sudo dnf groupinstall -y 'Development Tools'
```

Step 3 — Install flex and bison

```
sudo dnf install -y flex bison
```

For older `yum`-based systems (CentOS 7, RHEL 7) substitute `dnf` with `yum` in each command above.

3.3 Arch Linux / Manjaro (pacman)

```
sudo pacman -Syu
sudo pacman -S base-devel flex bison
```

3.4 openSUSE (zypper)

```
sudo zypper refresh
sudo zypper install -y flex bison gcc make
```

3.5 Build from Source (any distro)

Use this method to get the latest upstream versions of flex and bison.

Build flex from source

```
# Install dependencies first
sudo apt install -y autoconf automake libtool  # adjust for your distro

# Download and build flex
git clone https://github.com/westes/flex.git
cd flex
./autogen.sh
```

```
./configure --prefix=/usr/local
make -j$(nproc)
sudo make install
```

Build bison from source

```
# Download latest tarball from GNU FTP
wget https://ftp.gnu.org/gnu/bison/bison-3.8.2.tar.gz
tar -xzf bison-3.8.2.tar.gz
cd bison-3.8.2
./configure --prefix=/usr/local
make -j$(nproc)
sudo make install
```

4. Windows Installation

There are four supported methods for running flex and bison on Windows. Method A (WSL2) is the most recommended for development work.

Method	Best For
A — WSL2 (Windows Subsystem for Linux)	Developers wanting a full Linux environment on Windows
B — MSYS2 (MinGW-w64)	Native Windows binaries; closest to a Unix shell
C — Cygwin	Legacy compatibility; POSIX emulation layer
D — WinFlexBison (standalone)	Quick use without a full Unix environment; IDE integration

4A. Method A — WSL2 (Recommended)

Step 1 — Enable WSL2

Open PowerShell as Administrator and run:

```
wsl --install
# Restart your computer when prompted
```

Step 2 — Set WSL2 as default version

```
wsl --set-default-version 2
```

Step 3 — Install Ubuntu from Microsoft Store

Open the Microsoft Store, search for "Ubuntu 22.04 LTS", click Install, then launch it and create a user account.

Step 4 — Install flex and bison inside WSL2

Inside the WSL2 Ubuntu terminal, follow the Ubuntu/Debian instructions in Section 3.1:

```
sudo apt update
sudo apt install -y build-essential flex bison
```

> **Tip**
> You can access Windows files from WSL2 at /mnt/c/Users/<your-name>/. All Linux commands work natively.

4B. Method B — MSYS2 / MinGW-w64

Step 1 — Download and install MSYS2

Go to https://www.msys2.org/ and download the installer. Run it and accept defaults. MSYS2 installs to C:\msys64\ by default.

Step 2 — Update package database

Open "MSYS2 MSYS" from the Start menu and run:

```
pacman -Syu
# Close and reopen the terminal if prompted, then run again:
pacman -Su
```

Step 3 — Install toolchain and flex/bison

```
# For 64-bit native Windows binaries
pacman -S mingw-w64-x86_64-gcc flex bison make

# Add to PATH (optional; or use MSYS2 MinGW 64-bit shell)
export PATH=/mingw64/bin:$PATH
```

Step 4 — Verify

```
flex --version
bison --version
gcc --version
```

4C. Method C — Cygwin

Step 1 — Download Cygwin installer

Download setup-x86_64.exe from https://cygwin.com/install.html and run it.

Step 2 — Select packages during setup

In the package selection screen, search for and select:

- flex — under Devel category
- bison — under Devel category
- gcc-core — C compiler
- make — GNU make

Step 3 — Add Cygwin to PATH

Add C:\cygwin64\bin to your Windows system PATH environment variable, or always use commands from the Cygwin Terminal.

> **Note**
> Binaries produced by Cygwin depend on cygwin1.dll. They are not fully native Windows executables. For distribution, prefer MSYS2 or WinFlexBison.

4D. Method D — WinFlexBison (Standalone)

WinFlexBison provides pre-compiled win_flex.exe and win_bison.exe — no package manager required.

Step 1 — Download

Go to the GitHub Releases page: `https://github.com/lexxmark/winflexbison/releases` and download the latest **`win_flex_bison-X.X.X.zip`**.

Step 2 — Extract and place

Extract the ZIP to a permanent location such as C:\winflexbison\. The directory will contain win_flex.exe, win_bison.exe, and associated DLLs.

Step 3 — Add to PATH

1. Open Start Menu → search "Environment Variables" → click "Edit the system environment variables"
2. Click "Environment Variables" → under System Variables, find and select "Path" → click "Edit"
3. Click "New" and paste: C:\winflexbison
4. Click OK on all dialogs, then open a new Command Prompt and verify:

```
win_flex --version
win_bison --version
```

> **IDE Integration**
> WinFlexBison integrates well with Visual Studio. In project properties, add a Custom Build Step calling win_flex.exe on your .l file and win_bison.exe on your .y file.

5. Verifying Your Installation

5.1 Version Checks

Command	Expected Output (example)
flex --version	`flex 2.6.4`
bison --version	`bison (GNU Bison) 3.8.2`
gcc --version	`gcc (Ubuntu 11.x.x) 11.4.0`
win_flex --version (Windows D)	`win_flex 2.6.4`

5.2 Hello World — End-to-End Test

Create two files in the same directory to test the complete pipeline:

File: hello.l (flex rules)

```
%{
#include "hello.tab.h"
%}
%%
```

```
[0-9]+    { yylval = atoi(yytext); return NUMBER; }
"+"       { return PLUS; }
[ \t\n] { /* skip whitespace */ }
.         { return yytext[0]; }
%%
int yywrap() { return 1; }
```

File: hello.y (bison grammar)

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token NUMBER
%token PLUS
%%
expr:
    NUMBER PLUS NUMBER  { printf("Result: %d\n", $1 + $3); }
  | NUMBER              { printf("Number: %d\n", $1); }
  ;
%%
void yyerror(const char *s) { fprintf(stderr, "Error: %s\n", s); }
int main() { return yyparse(); }
```

Build commands (Linux / WSL2 / MSYS2 / Cygwin)

```
bison -d hello.y          # produces hello.tab.c and hello.tab.h
flex hello.l              # produces lex.yy.c
gcc -o hello lex.yy.c hello.tab.c -lfl

# Test it
echo '3 + 5' | ./hello    # Expected: Result: 8
```

Build commands (Windows — WinFlexBison + MinGW/MSVC)

```
win_bison -d hello.y
win_flex hello.l
gcc -o hello.exe lex.yy.c hello.tab.c

echo 3 + 5 | hello.exe
```

Note on -lfl flag

On some systems you may need -ll instead of -lfl (original lex library). On others, the library is already embedded. If you get a linker error, try omitting the flag or substituting -ll.

6. Common Errors & Solutions

Error / Symptom	Solution
flex: command not found	Install flex using your distro's package manager (Section 3) or verify PATH includes the install directory
bison: command not found	Same as above for bison
undefined reference to `yywrap'	Add int yywrap(){return 1;} to your .l file, or compile with -lfl / -ll flag
undefined reference to `yylex'	Ensure lex.yy.c is included in the gcc command alongside y.tab.c
conflicts: X shift/reduce	Grammar is ambiguous. Review operator precedence with %left, %right, %nonassoc in your .y file
win_flex not recognized (Windows)	Add C:\winflexbison to the system PATH (Section 4D, Step 3) and reopen the terminal
WSL2 not available (Windows 10 older builds)	Ensure Windows 10 version 1903 (build 18362) or later. Run: wsl --update in Admin PowerShell
cygwin1.dll missing	Add C:\cygwin64\bin to PATH, or copy cygwin1.dll alongside your executable

7. Quick Reference — Key Files & Commands

7.1 Generated Files

File	Description
lex.yy.c	C source generated by flex from your .l file
y.tab.c	C source for the parser generated by bison/yacc
y.tab.h	Header file with token definitions (use #include in .l file)
y.output	Human-readable parser state table (generated with bison -v)

7.2 Essential Flags

Command	Effect

flex -o mylex.c input.l	Write output to custom filename instead of lex.yy.c
bison -d grammar.y	Generate both grammar.tab.c and grammar.tab.h
bison -v grammar.y	Also produce grammar.output (state machine debug info)
bison -Wall grammar.y	Enable all bison warnings
gcc ... -lfl	Link against the flex library (provides yywrap default)

8. Additional Resources

- GNU Flex Manual: `https://westes.github.io/flex/manual/`
- GNU Bison Manual: `https://www.gnu.org/software/bison/manual/`
- WinFlexBison Releases: `https://github.com/lexxmark/winflexbison/releases`
- MSYS2 Home: `https://www.msys2.org/`
- WSL2 Documentation: `https://learn.microsoft.com/en-us/windows/wsl/`
- "Compilers: Principles, Techniques and Tools" (Aho, Lam, Sethi, Ullman) — the definitive compiler textbook
- "flex & bison" by John Levine (O'Reilly) — the most practical book on the toolchain

—— End of Guide ——